

04 — Hierarquia de Memória

Pré-requisito: [03-simd.md](#)

1. Intuição: memória rápida é cara e pequena; memória grande é barata e lenta

Existe uma tensão física fundamental que nenhum avanço de engenharia resolveu de verdade: memória que responde rapidíssimo (poucos átomos de distância do núcleo de processamento, feita de transistores caros) só cabe em pequena quantidade no chip. Memória que cabe em grande quantidade (gigabytes) precisa de tecnologia mais barata, fisicamente mais distante, e portanto mais lenta de alcançar.

A solução da indústria inteira, há décadas, é a mesma: em vez de escolher um ponto só nesse trade-off, **empilhar vários níveis**, cada um maior e mais lento que o anterior, e deixar o hardware decidir automaticamente o que fica em cada nível, baseado no que foi usado recentemente.

2. Os níveis, do mais rápido ao mais lento

REGISTRADORES	alguns, dentro da própria ULA	~0 (parte do mesmo ciclo)
L1 CACHE	~32-64 KB, por núcleo	~1 ns (~4 ciclos)
L2 CACHE	~256KB-1MB, por núcleo ou par	~4 ns (~12 ciclos)
L3 CACHE	~8-32 MB, compartilhado entre núcleos	~15 ns (~40 ciclos)
RAM (DRAM)	gigabytes	~100 ns (~300 ciclos)

Os números exatos variam por CPU, mas a **forma** da pirâmide — cada nível ~10x maior e ~4-10x mais lento que o anterior — é universal em hardware moderno, CPU e GPU.

PERGUNTA RÁPIDA 1: se acessar RAM leva ~100ns e um acesso a L1 leva ~1ns, quantas operações de L1 "cabem" no tempo de UM acesso a RAM?

► Resposta

3. Por que cache funciona: localidade

Cache só é útil porque programas reais têm **localidade** — a tendência de reusar dado recentemente acessado, ou acessar dado fisicamente perto do que acabou de ser acessado. Dois tipos, e os dois importam:

- **Localidade temporal:** se você usou um dado agora, é provável que use ele de novo em breve (uma variável de loop, por exemplo). Cache guarda o que foi usado recentemente, apostando nisso.
- **Localidade espacial:** se você acessou um endereço, é provável que acesse endereços **vizinhos** em breve (percorrer um array). É por isso que cache não busca 1 byte por vez — busca uma **cache line** inteira (tipicamente 64 bytes) de cada vez, trazendo os vizinhos "de graça" na aposta de que você vai precisar deles.

Isso não é novidade pra você — é exatamente o mecanismo que expliquei hoje mais cedo, quando comparamos array contíguo (C++) com lista espalhada (Python): a cache line de 64 bytes é literalmente a unidade que faz "dado vizinho na memória" ser rápido de ler em sequência.

4. Cache miss: o preço de apostar errado

Quando o dado que você pede **não** está em nenhum nível de cache, isso é um **cache miss** — a CPU precisa ir até o próximo nível (ou até a RAM, no pior caso), pagando a latência cheia daquele nível. Um **cache hit** é quando o dado já estava lá, pago o preço barato.

Acesso sequencial a um array grande (percorrendo em ordem):

```
arr[0]  -> MISS (busca a cache line inteira de 64 bytes, traz arr[0..15] se forem floats)
arr[1]  -> HIT  (já estava na cache line trazida)
arr[2]  -> HIT
...
arr[15] -> HIT
arr[16] -> MISS de novo (nova cache line)
```

Um miss a cada 16 acessos (pra floats de 4 bytes, cache line de 64 bytes) é excelente — 16x mais barato, em média, que buscar cada elemento individualmente na RAM.

5. Acesso com stride: onde a localidade espacial quebra

```
// stride 1 (sequencial) -- localidade espacial ótima
for (int i = 0; i < n; i++) soma += arr[i];

// stride 16 -- pula 16 posições a cada acesso
for (int i = 0; i < n; i += 16) soma += arr[i];
```

Com stride 16 (pra floats de 4 bytes, isso é 64 bytes de salto — exatamente uma cache line inteira por acesso), **cada** acesso vira um cache miss novo, porque você nunca reaproveita o resto da linha que acabou de ser trazida. É o mesmo padrão de "lista espalhada" que discutimos sobre Python, só que provocado deliberadamente com stride, em C++.

PERGUNTA RÁPIDA 2: acessar um array `struct Particle { float x,y,z,vx,vy,vz; } particles[N]` lendo só o campo `x` de cada elemento (lembra do Módulo 08, AoS?) tem que tipo de padrão de acesso — stride 1, ou stride maior? Calcule o stride em bytes.

► Resposta

6. Cache no contexto de múltiplos núcleos: coerência (breve, não aprofundado aqui)

Quando dois núcleos de CPU têm cada um sua própria L1/L2, e ambos guardam uma cópia do mesmo endereço de memória, o hardware precisa garantir que, se um núcleo escreve, o outro não continue lendo um

valor velho da própria cache. Isso se chama **coerência de cache**, e é resolvido por um protocolo de hardware (fora do escopo aqui — mencionado apenas porque você vai ouvir o termo, e porque "cada thread tem sua própria visão de cache" é uma simplificação, não a regra exata).

7. Por que isso importa para CUDA

Isso é praticamente uma tradução direta:

CPU	GPU (nome que você vai encontrar na Phase 2)
Registrador	Registrador (por thread)
L1 cache	Shared memory / L1 (por SM)
L2 cache	L2 cache (compartilhado entre SMs)
RAM	Memória global (VRAM)

A ideia central — "memória perto é rápida e pequena, memória longe é lenta e grande, e localidade decide se você paga o preço caro ou o barato" — é **idêntica**. Memory coalescing (que você já discutiu bastante hoje) é, no fundo, a versão GPU de "acesso sequencial aproveita a cache line inteira": quando 32 threads de um warp leem endereços vizinhos, o hardware consegue combinar isso numa transação só, exatamente como uma cache line de CPU trazendo vários floats vizinhos de uma vez.

8. Erros comuns

- Achar que "mais cache é sempre melhor". O tamanho é fixo pelo hardware que você tem — o que você controla é o **padrão de acesso** do seu código, pra aproveitar o que já existe.
- Ignorar stride como fonte de lentidão silenciosa — código que "faz o mesmo número de operações" pode ser 10x mais lento só por causa do padrão de acesso, sem nenhuma mudança na lógica.
- Achar que cache miss trava o programa. Não trava — só é mais lento. É invisível no código-fonte, só aparece em número de tempo de execução ou em profiler.

PREDICT BEFORE RUNNING (conceitual)

Um array de 1 milhão de floats (4 MB), numa CPU com L1 de 32 KB, L2 de 256 KB, L3 de 8 MB.

1. O array inteiro cabe em L1? Em L2? Em L3?
2. Percorrer esse array sequencialmente uma única vez — a maioria dos acessos vai ser hit ou miss, considerando que o array não cabe nos níveis mais rápidos?
3. Percorrer o **mesmo** array 1000 vezes seguidas, num loop externo — a partir da segunda passada, o comportamento de cache muda? Por quê?

► Explicação